Multithreaded algorithms: solutions

27-E01 Work, span, and processor lower bounds

Derivation

Lower bound max(ceil(W/P), D) gives 90 at P8 and 45 at P16. Strong greedy upper bound(W−D)/P+D gives 696/8+24=111 and 696/16+24=67.5. Integral unit time sharpens the latter to T≤67. These are intervals, not exact times.

Parallelism720/24=30 bounds speedup, not marginal processor benefit above30. In the specified DAG, the independent stage takes ceil(697/P), then the chain 23; the stages cannot overlap. Full-batch execution attains that lower bound. Times are 24+23=47 at P30 and 23+23=46 at P31.

Conclusion and checks

Greedy intervals are[90,111] and[45,67]. The 31st processor helps on the stated DAG while speedup720/46 remains below30.

27-E02 Recursive matrix work and dependencies

Derivation

Work sums all eight products:$W(n)=8W(n/2)+\Theta(n^2)$, with $8^{(\log_2 n)}=n^3$ leaves, giving $\Theta(n^3)$. Span follows one child plus $\Theta(\log n)$ combination. For n=2^L, the level costs $\Theta(L),\Theta(L-1),\ldots$ sum to $\Theta(L^2)$.

The factor8 repeats at every recursive level, not merely once at the root. Removing it changes the entire tree. Nonatomic C11+=ae and C11+=bg share every read-modify-write destination; the other three blocks have the same conflict.

Conclusion and checks

Work $\Theta(n^3)$, span $\Theta(\log^2 n)$, parallelism $\Theta(n^3/\log^2 n)$. Separate C, T destinations followed by synchronized addition remove the race.

27-E03 Worst-case parallel-merge span

Derivation

Let n1≥n2, N=n1+n2. Either side of the longer median has at most floor(n1/2) records. Even if all second-run records go to one side, child size≤n1/2+n2=(N+n2)/2≤3N/4. Binary search costs O(1+log(n2+1))≤O(log(N+1)). Constant-factor shrinkage gives O(log N) levels whose decreasing logarithmic sizes sum to $O(\log^2(N+1))$.

Multithreaded algorithms: solutions (continued)

Equal-length power-of-two even and odd runs remain the same interleaved form in the left child. The other run stays proportional to current size and a middle cut takes Θ(log size) comparisons. Their sum is $\Omega(\log^2 N)$. An always-empty other run makes each search constant, leaving Θ(log(N+1)) median-tree span.

Conclusion and checks

Worst span is $\Theta(\log^2 N)$, but not every input attains it. Counting recursion levels without paying their searches misses a logarithmic factor.

27-E04 Scheduling nine Fibonacci strands

Derivation

The root has pre3, cont3, join3. Left P-FIB2 has pre2, cont2, join2, F1a, F0. The root's right child is F1b. Dependencies are pre3→{pre2, cont3}, cont3→F1b, pre2→{F1a, cont2}, cont2→F0,{F1a, F0}→join2,{join2, F1b}→join3.

Step	Strands on P=3
1	pre3
2	pre2, cont3
3	F1a, cont2, F1b
4	F0
5	join2
6	join3

Lin Cing https://github.com/xinhi6716/computer-science-self-study

Work 9 and path pre3, pre2, cont2, F0, join2, join3 give span6, parallelism3/2. Every step executes all ready strands, so this is greedy and attains the span lower bound, proving optimum6.

Set Zn=Wn+3. Then Zn=Z(n−1)+Z(n−2), Z0=Z1=4, so Zn=4F(n+1), Wn=4F(n+1)−3. Bases D0=D1=1, D2=4 and recurrence2+max(D(n−1),1+D(n−2)) give D3=6, then induction gives Dn=2n.

Conclusion and checks

P-FIB3 has W9, D6, and P3 optimum6 steps. At n4, W17, D8 matches the same model.

Multithreaded algorithms: solutions (continued)

27-E05 Lost updates and two repairs

Derivation

Execute T1 load0 to r1, T2 load0 to r2, T1 add to 1 and store1, T2 add to 1 and store1. Both internal instruction orders hold, with final x=1. Final sync only waits; it cannot restore the overwritten increment.

Truly atomic additions to one cell produce the right sum but serialize n dependent cell operations in this model, giving Θ(n) span. Separate task-i writes to v[i] followed by balanced reduction give Θ(n) work,Θ(log(n+1)) span. Each parent adds two finished local values. This requires associative addition and independent storage; floating-point reassociation requires a separate rounding assessment.

Conclusion and checks

Sync does not atomize prior compound operations. Atomic addition and reduction both repair the answer but create different dependencies and spans.

27-E06 Reduction for rectangular matrices

Derivation

Use nested parallel loops for i∈[0, p), j∈[0, r), with an independent q-term dot product per cell. DOT(klo, khi) returns0 if empty, one product A[i, k]B[k, j] if singleton, otherwise forks its lower half, calls its upper half, syncs, and returns their sum. Only the DOT root for(i, j) writes Cij.

A DOT uses q multiplications, q−1 additions,Θ(q) work, and Θ(1+log q) span. Outer issuing work Θ(pr) is within Θ(pqr). Total span is Θ(1+log p+log r+log q), remaining Θ(1) for p=q=r=1 instead of falsely claiming zero work from log1.

With every reduction tree fully expanded concurrently, up to Θ(pqr) task descriptors/local intermediates can be live. Input pq+qr and output pr cells are additional. Depth-first or bounded-P execution can reduce live workspace but does not establish the same peak for every schedule. Allocation uses uninitialized storage as specified; serial initialization would add cost.

Conclusion and checks

Work Θ(pqr), span Θ(1+log p+log r+log q), without concurrent accumulation into one cell. All unit-dimension cases remain covered.

Multithreaded algorithms: solutions (continued)

27-E07 Longer-run swaps and tie rules

Derivation

Key-only comparison swaps to first run[1b,2c], second[1a]. The upper median is 2c; lower_bound in[1a] returns the end, so O2=2c. The left child merges[1b],[1a]. Pivot1b finds the first position because1a≥1, so O0=1b and the remaining1a goes to O1. Result[1b,1a,2c] is unstable.

With(key, original position),1a<1b<2c. The second search passes1a, placing1b at O1 and 1a at O0. Descriptor swaps do not change record identities or their fixed total order. Induction therefore preserves equal-key original order.

Conclusion and checks

Key-only merge stays key-sorted but loses stability. Unique original identities restore[1a,1b,2c]. Temporary run roles cannot replace original order.

27-E08 Output offsets and aliasing

Derivation

Root m=2, pivot6c. The other cut is j=6, since3f<6c<6g. Thus q=3+(2−0)+(6−4)=7. Left output[3,7) has 4 records, pivot is 7, and right output[8,11) has 3.

Final O[3..11)=[1a,2e,3f,4b,6c,6g,8h,9d]. Other cells remain untouched. Offset o adds to relative ranks without changing input cuts.

If O=T, o=0, the root writes6c over T4=2e. Its left child still needs original T[4..6) containing2e,3f. The first value is already lost, and the run may no longer be sorted. This procedure is not an in-place merge.

Conclusion and checks

Root q7 and child ranges[3,7),[8,11). Disjoint output ranges alone do not prove safety; output must also avoid unread input.

27-E09 General balanced-tree charging

Derivation

At size m, divide c log(m+1) among its m records. A fixed record pays until selected as pivot. Counting r levels upward, size is at leastρ^(−r). Since g(u)=log(u+1)/u decreases, the charge is at most cρ^r log(ρ^(−r)+1)≤cρ^r[1+r log(1/ρ)].

Multithreaded algorithms: solutions (continued)

Both geometric sums Σρ^r,Σrρ^r converge for fixedρ, so each record pays O(1) with aρ-dependent constant, and all records O(N). If every node creates at most a fixed number of children, and each nonempty call removes one record, there are O(N) empty calls. Binary merge has exactly N+1. Without a child-count bound, empty-call count itself can be arbitrarily large and needs an additional control-cost assumption.

For a chain removing one record and putting m−1 into one child, the ratio1−1/m approaches1. Costs log(m+1) sum to log((N+1)!)=Θ(N log N). A fixed-ρ convergence constant no longer applies.

Conclusion and checks

Fixed shrinkage, fixed maximum child count, and one unique removed record per call give O(N) total work including empty calls. Charging cannot make unrestricted empty-call overhead disappear.

27-E10 One input family must realize worst sort span

Derivation

Individually expensive merges at each level do not imply one compatible input forces all of them. For n16, use4-bit reversal:[0,8,4,12,2,10,6,14,1,9,5,13,3,11,7,15]. Root halves sort to[0,2,4,6,8,10,12,14] and[1,3,5,7,9,11,13,15].

Divide the even first half by 2 to obtain the n8 relative-rank reversal; subtract1 and divide2 in the odd half to obtain the same. This persists recursively. Every merge therefore receives near-equal interleaved runs with Ω(log^2(size)) span. Parent merge starts after both children, so the longest child path and a longest parent-merge path concatenate.

Conclusion and checks

The sumΣ_(r=1..L)r^2=Θ(L^3) matches the upper bound, giving worst Θ(log^{3}n). For n16, the square sum4^2+3^2+2^2+1^2=30 illustrates the series; it is not exact execution time without fixed operation constants.

27-E11 Reversing the work/span preference

Derivation

A sufficient guarantee for B faster is 1536/P+32<2048/P, or P<16. Thus integer 1≤P≤15 works. A sufficient guarantee for A faster is 2048/P+8<32, or P>256/3, so P≥86 works.

Multithreaded algorithms: solutions (continued)

For $16 \le P \le 85$, these sufficient conditions fail. That neither proves equal times nor permits subtracting upper bounds as actual time differences. Stronger greedy bounds or actual DAG/schedule information may narrow uncertainty; the specified loose bounds give no decision there.

Conclusion and checks

These ranges are sufficient guarantees, not a claim of maximal possible ranges. Lower work with larger span can reverse preference as P changes.

27-E12 Serial spawning differs from parallel for

Derivation

Let r=ceil(n/g). Ordinary-loop spawning has a $\Theta(r)$ serial parent chain, and the largest batch costs $\Theta(g)$. Span is $O(r+g)$ and at least $\Omega(\max(r, g))=\Omega(r+g)$, hence $\Theta(n/g+g)$. Work is $\Theta(n+r)=\Theta(n)$.

Since $n/g+g \ge 2\sqrt{n}$, an integer g near $\sqrt{n}$ achieves best $\Theta(\sqrt{n})$ span and parallelism. Balanced issuance needs only $\Theta(\log r)$ control depth, giving $\Theta(\log r+g)$. In the ideal constant-cost model, g=1 gives $\Theta(\log n)$, though real fine-task constants may favor a coarser fixed grain.

Conclusion and checks

Task issuance itself changes span. Ordinary-loop spawn is not PAR-FOR. Equal linear work does not imply equal critical paths.

27-E13 Matrix multiplication with less temporary storage

Derivation

Pad to a power of two, and keep C disjoint from read-only A, B. Parallel-zero C before ACC.

Multithreaded algorithms: solutions (continued)

```
ACC(C, A, B):
    n=A.rows
    if n == 1: C[0,0]+=A[0,0]*B[0,0]; return
    view A as [[a, b],[c, d]], B as [[e, f],[g, h]]
    view C into C11, C12, C21, C22
    spawn ACC(C11, a, e); spawn ACC(C12, a, f)
    spawn ACC(C21, c, e); ACC(C22, c, f)
    sync
    spawn ACC(C11, b, g); spawn ACC(C12, b, h)
    spawn ACC(C21, d, g); ACC(C22, d, h)
    sync
```

Within each phase, four children update distinct C blocks. Sync between phases finishes the first contribution before the second to the same block. Induction gives correct block contributions and final original C+AB.

Work $W(n)=8W(n/2)+\Theta(1)=\Theta(n^3)$; initial $\Theta(n^2)$ zeroing is lower order. Span $D(n)=2D(n/2)+\Theta(1)=\Theta(n)$; initial $\Theta(\log n)$ zeroing is lower order. There is no additional $n\times n$ numeric temporary. Call/task descriptors peak at $O(n^2)$, one path uses $O(\log n)$ stack, and A, B, C occupy $\Theta(n^2)$ cells separately.

Conclusion and checks

Parallelism $\Theta(n^2)$ is below eight-way $\Theta(n^3/\log^2 n)$, but avoids the literal full-expansion $\Theta(n^3)$ workspace. The saved space comes from two-stage dependencies and cannot retain the original $\log^2$ span.

27-E14 Two meanings of average random-DAG speedup

Derivation

The first DAG can be10 independent length 10 chains:W100, D10, and T10=10. The second is a length 100 chain and T10=100. Equal probabilities give E[W/T10]= (10+1)/2=11/2, whereas E[W]/E[T10]=100/55=20/11.

The first is mean per-run speedup; the second is the ratio of accumulated work to accumulated time over a fixed batch. Running each once gives work 200, time 110, and ratio20/11. They are distinct statistics.

For every sampled DAG, pointwise bounds are $T_P\geq W/P$, $T_P\geq D$, and greedy $T_P\leq (W-D)/P+D$. Taking expectations and using linearity gives

$$\max\{E[W]/P, E[D]\} \leq E[T_P] \leq E[W]/P + (1-1/P)E[D].$$

Multithreaded algorithms: solutions (continued)

Conclusion and checks

No independence between W, D is needed. Only pointwise inequalities and linearity are used. For aggregate-work speedup, use E[W]/E[Tp], with E[W]/E[D] as the corresponding upper bound.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Multithreaded algorithms: solutions (continued)

Verification code appendix: parallel_depth_checks.py

```
"""Executable finite models accompanying the general proofs; not a timing benchmark."""
from itertools import combinations, combinations_with_replacement, product
from functools import lru_cache
from fractions import Fraction
from random import Random

SAMPLE=[(1,0),(4,1),(6,2),(9,3),(2,4),(3,5),(6,6),(8,7)]

def fib_dag(n,prefix=''):
    if n<2:
        name=prefix+'b';return {name:f'F{n}'},[],name,name
    L,le,ls,lt=fib_dag(n-1,prefix+'L');R,re,rs,rt=fib_dag(n-2,prefix+'R')
    p,c,j=(prefix+s for s in ('p','c','j'))
    return {**L,**R,p:f'pre{n}',c:f'cont{n}',j:f'join{n}'},le+re+[(p,ls),(p,c),(c,rs),(lt,j),
(rt,j)],p,j

def span(nodes,edges):
    pred={v:set() for v in nodes}
    for u,v in edges:pred[v].add(u)
    @lru_cache(None)
    def depth(v):return 1+max((depth(u) for u in pred[v]),default=0)
    return max((depth(v) for v in nodes),default=0)

def greedy(nodes,edges,P,reverse=False):
    pred={v:set() for v in nodes}
    for u,v in edges:pred[v].add(u)
    done=set();steps=[]
    while len(done)<len(nodes):
        ready=sorted((v for v in nodes if v not in done and pred[v]<=done),reverse=reverse)
        assert ready
        batch=ready[:P];steps.append(batch);done.update(batch)
    return steps

def merge(T,l1,r1,l2,r2,O,o,trace=None,key=lambda x:x,right_first=False):
    """Same half-open ranges and separate-output contract as the text."""
    assert T is not O
    if trace is None:trace=[]
    if r1-l1<r2-l2:l1,r1,l2,r2=l2,r2,l1,r1
    n1,n2=r1-l1,r2-l2
    if not n1:
        trace.append({'empty':True});return trace
    m=(l1+r1)//2;x=T[m];lo,hi=l2,r2;search=[]
    while lo<hi:
        mid=(lo+hi)//2;search.append((lo,hi,mid,T[mid]))
        if key(T[mid])<key(x):lo=mid+1
        else:hi=mid
    j=lo;q=o+(m-l1)+(j-l2)
    left=m-l1+j-l2;right=r1-m-1+r2-j
    assert left+right==n1+n2-1 and 4*max(left,right)<=3*(n1+n2)
```

Lin Cing https://github.com/xinhi6716/computer-science-self-study

Multithreaded algorithms: solutions (continued)

```
    O[q]=x

trace.append(dict(empty=False,l1=l1,r1=r1,l2=l2,r2=r2,m=m,j=j,q=q,pivot=x,N=n1+n2,search=sear
ch))
    args=[(l1,m,l2,j,o),(m+1,r1,j,r2,q+1)]
    for a,b,c,d,out in args[::-1] if right_first else args:
        merge(T,a,b,c,d,O,out,trace,key,right_first)
    return trace

def sort(values,right_first=False):
    original=[(key,i) for i,key in enumerate(values)];X=original.copy();Y=original.copy()
    def go(src,dst,l,r):
        assert src[l:r]==original[l:r] and dst[l:r]==original[l:r]
        if r==l:return
        if r-l==1:dst[l]=src[l];return
        m=(l+r)//2;calls=[(l,m),(m,r)]
        for a,b in calls[::-1] if right_first else calls:go(dst,src,a,b)
        merge(src,l,m,m,r,dst,l,right_first=right_first)
    go(X,Y,0,len(X));return Y

def dot(a,b):
    assert len(a)==len(b)
    if not a:return 0,0,0
    if len(a)==1:return a[0]*b[0],1,1
    m=len(a)//2;x,w,d=dot(a[:m],b[:m]);y,W,D=dot(a[m:],b[m:])
    return x+y,w+W+1,max(d,D)+1

def multiply(A,B):
    n=len(A)
    if n==1:return [[A[0][0]*B[0][0]]]
    h=n//2
    def part(M,i,j):return [row[j:j+h] for row in M[i:i+h]]
    C=[[0]*n for _ in range(n)];T=[[0]*n for _ in range(n)]
    for i,j in product((0,h),repeat=2):
        first=multiply(part(A,i,0),part(B,0,j));second=multiply(part(A,i,h),part(B,h,j))
        for u,v in product(range(h),repeat=2):C[i+u][j+v]=first[u][v];T[i+u][j+v]=second[u]
[v]
    return [[C[i][j]+T[i][j] for j in range(n)] for i in range(n)]

def accumulate(C,A,B):
    n=len(A)
    if n==1:C[0][0]+=A[0][0]*B[0][0];return
    h=n//2
    def part(M,i,j):return [row[j:j+h] for row in M[i:i+h]]
    for k in (0,h):
        for i,j in product((0,h),repeat=2):
            block=part(C,i,j);accumulate(block,part(A,i,k),part(B,k,j))
            for u,v in product(range(h),repeat=2):C[i+u][j+v]=block[u][v]

def bit_reverse(n):
```

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Multithreaded algorithms: solutions (continued)

```
    k=n.bit_length()-1;assert n==1<<k
    return [int(f'{i:0{k}b}'[::-1],2) if k else 0 for i in range(n)]

def interleaved_worst(n):
    def go(keys):
        if len(keys)<2:return keys
        first=keys[::2] if len(keys)%2==0 else keys[1::2]
        second=keys[1::2] if len(keys)%2==0 else keys[::2]
        return go(first)+go(second)
    return go(list(range(n)))

def check():
    result={};fib=[0,1]
    for _ in range(13):fib.append(sum(fib[-2:]))
    for n in range(12):
        nodes,edges,_,_=fib_dag(n);W=len(nodes);D=span(nodes,edges)
        assert W==4*fib[n+1]-3 and D==(1 if n<2 else 2*n)
    N,E,_,_=fib_dag(3);assert len(greedy(N,E,3))==6
    result['fibonacci_sizes']=12
    dag_cases=0
    for n in range(1,6):
        possible=list(combinations(range(n),2))
        for mask in range(1<<len(possible)):
            edges=[e for i,e in enumerate(possible) if mask>>i&1];D=span(range(n),edges)
            for P in range(1,n+1):
                for reverse in (False,True):
                    t=len(greedy(range(n),edges,P,reverse))
                    assert max((n+P-1)//P,D)<=t<=Fraction(n-D,P)+D
                    dag_cases+=1
    result['greedy_small_dag_schedules']=dag_cases
    assert (697+29)//30+23==47 and (697+30)//31+23==46
    cases=0
    runs=[a for n in range(6) for a in combinations_with_replacement(range(3),n)]
    for a,b in product(runs,repeat=2):
        T=[(v,i) for i,v in enumerate(a+b)];n=len(T)
        for rev in (False,True):
            O=[None]*(n+6);trace=merge(T,0,len(a),len(a),n,O,3,right_first=rev)
            assert O[:3]==[None]*3 and O[n+3:]==[None]*3 and O[3:n+3]==sorted(T)
            writes=[s['q'] for s in trace if not s['empty']]
            assert len(trace)==2*n+1 and sorted(writes)==list(range(3,n+3));cases+=1
    result['stable_merge_range_and_serialization_cases']=cases
    O=[None]*8;trace=merge(SAMPLE,0,4,4,8,O,0)
    assert [s['q'] for s in trace if not s['empty']]==[4,3,2,1,0,6,5,7]
    assert trace[0]['j']==6 and trace[0]['m']==2 and O==sorted(SAMPLE)
    T=[(1,0),(1,1),(2,2)];O=[None]*3;merge(T,0,1,1,3,O,0,key=lambda v:v[0]);assert O==[(1,1),
(1,0),(2,2)]
    arrays=0
    for n in range(9):
```

Multithreaded algorithms: solutions (continued)

```
        for values in product(range(3),repeat=n):
            for rev in (False,True):assert sort(values,rev)==sorted((v,i) for i,v in
enumerate(values))
            arrays+=1
    result['two_buffer_stable_sort_arrays_both_orders']=arrays
    for k in range(1,9):
        n=1<<k;v=bit_reverse(n)
        assert sorted(v[:n//2])==list(range(0,n,2)) and sorted(v[n//2:])==list(range(1,n,2))
    result['bit_reversal_dyadic_sizes']=8
    for n in range(1,130):
        v=interleaved_worst(n)
        assert sort(v)==[(i,v.index(i)) for i in range(n)]
        def inspect(a):
            if len(a)<2:return
            h=len(a)//2;left=sorted(a[:h]);right=sorted(a[h:]);order=sorted((x,0) for x in
left)+sorted((x,1) for x in right)
            origins=[side for _,side in sorted(order)]
            assert all(origins[i]!=origins[i+1] for i in range(len(origins)-1))
            inspect(a[:h]);inspect(a[h:])
        inspect(v)
    result['all_levels_interleaved_nondyadic_sizes']=129
    outcomes=set();interleavings=0
    for slots in combinations(range(6),3):
        order=[0 if t in slots else 1 for t in range(6)];pc=[0,0];reg=[0,0];x=0
        for who in order:
            if pc[who]==0:reg[who]=x
            elif pc[who]==1:reg[who]+=1
            else:x=reg[who]
            pc[who]+=1
        outcomes.add(x);interleavings+=1
    assert outcomes=={1,2};result['two_task_load_add_store_interleavings']=interleavings
    assert dot([3,-1,2,4],[2,5,-3,1])==(-1,7,3)
    rng=Random(2704);matrices=0
    for n in (1,2,4,8):
        for _ in range(20):
            A=[[rng.randrange(-5,6) for _ in range(n)] for _ in range(n)];B=
[[rng.randrange(-5,6) for _ in range(n)] for _ in range(n)]
            AB=[[sum(A[i][k]*B[k][j] for k in range(n)) for j in range(n)] for i in range(n)]
            assert multiply(A,B)==AB
            C=[[rng.randrange(-5,6) for _ in range(n)] for _ in range(n)];old=[r[:] for r in
C];accumulate(C,A,B)
            assert C==[[old[i][j]+AB[i][j] for j in range(n)] for i in range(n)];matrices+=1
    result['eight_product_and_two_phase_matrix_pairs']=matrices
    assert Fraction(10+1,2)==Fraction(11,2) and Fraction(100,55)==Fraction(20,11)
    return result

if __name__=='__main__':
    import json
    print(json.dumps(check(),indent=2))
```

Lin Cing https://github.com/xinhi6716/computer-science-self-study

Multithreaded algorithms: solutions (continued)

Verification code appendix: work_span_bounds.py

```
"""Compute work, span, parallelism, and processor lower bounds."""

def bounds(work, span, processors):
    assert work >= span > 0 and processors > 0
    return {
        "work": work,
        "span": span,
        "parallelism": work / span,
        "processor_lower_bound": max(work / processors, span),
    }


assert bounds(720, 24, 8)["processor_lower_bound"] == 90
assert bounds(720, 24, 16)["processor_lower_bound"] == 45
assert bounds(1000, 50, 32)["processor_lower_bound"] == 50
print(bounds(720, 24, 16))
```

References and attribution

Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, Clifford Stein. Introduction to Algorithms, 3rd Edition.

Original educational synthesis by Lin Cing. Original material is licensed under CC BY 4.0. The referenced textbook is not distributed with these notes.

https://github.com/xinhi6716/computer-science-self-study